IPsec e Transport Layer Security

Laboratorio del corso “Sicurezza dei sistemi informativi” (01UDUOV)
Politecnico di Torino – AA 2023/24
Prof. Cataldo Basile

preparata da:
Gabriele Gatti (gabriele.gatti@polito.it)

v. 1.1 (12/12/2023)

Indice

1 Scopo dell’esercitazione **2**
1.1 Comandi accessori . . . 2
1.2 Comandi utili: IPsec e sicurezza a livello IP . . . 3
1.2.1 IPsec Security Associations (ip xfrm state) . . . 3
1.2.2 IPsec Security Policies (ip xfrm policy) . . . 4

2 IPSec e la sicurezza a livello IP **6**
2.1 Definizione di politiche e associazioni IPsec . . . 6
2.2 Modifica di politiche e associazioni IPsec . . . 8
2.3 Misure di prestazione . . . 13
2.4 Il protocollo IKE . . . 13
2.4.1 strongSwan e pre-shared-keys . . . 13

3 Il protocollo TLS **16**
3.1 Instaurare un canale TLS . . . 16
3.2 Configurazione di un server TLS . . . 17
3.3 Uso del Session ID . . . 18
3.4 Autenticazione del client tramite TLS . . . 18
3.5 Attacco MITM a TLS . . . 19
3.6 Misure di prestazione . . . 20

4 Approfondimenti ed ulteriori esercizi (opzionali) **21**
4.1 strongSwan e certificati X.509 . . . 21
4.2 Connessione sicura gateway-to-gateway . . . 24

1 Scopo dell'esercitazione

Scopo di questa esercitazione è sperimentare la creazione di canali di comunicazione sicura tra due sistemi di elaborazione, valutarne le caratteristiche di sicurezza e misurarne le prestazioni al variare della configurazione. In particolare, gli esperimenti riguardando l'uso di IPsec per la protezione a livello IP e TLS per la protezione di canali TCP.

IPsec (IP Security) è la proposta IETF (`www.ietf.org`) per garantire la sicurezza nelle reti IP, sia in IPv4 sia in IPv6. L'idea fondamentale è di fornire una serie di servizi comuni di sicurezza al livello rete per evitare di dover duplicare queste funzionalità in ogni applicazione. IPsec offre essenzialmente autenticazione, integrità e riservatezza dei pacchetti IP. IPsec permette ad un sistema di definire le proprie politiche di sicurezza, di selezionare i protocolli appropriati per le politiche scelte, di specificare gli algoritmi crittografici usati, e di acquisire le chiavi crittografiche necessarie. Per comunicare in modalità IPsec, due nodi devono condividere almeno una Security Association (SA) per specificare tutte le caratteristiche di sicurezza del collegamento. Le SA IPsec possono essere specificate manualmente o negoziate (in modo sicuro) attraverso il protocollo IKE.

La sperimentazione di IPsec è basata sull'uso della sua implementazione e dei relativi strumenti di configurazione disponibili sulla piattaforma Linux:

- `ip xfrm` è il tool per manipolare le politiche e SA IPsec (vedi `man ip xfrm`);
- strongSwan è il demone per la gestione del protocollo IKE e supporta l'autenticazione con Pre-Shared Keys (PSK), certificati X.509 e Kerberos (`man ipsec`);
- il file `/etc/ipsec.conf` definisce il comportamento di strongSwan (`man ipsec.conf`).

La seconda parte dell'esercitazione prevede la sperimentazione del protocollo TLS (Transport Layer Security) per la protezione a livello trasporto. Sarà usata la libreria OpenSSL (e alcuni tool associati) che offre un supporto completo alla configurazione e sperimentazione di canali SSL/TLS. In particolare, il comando `s_client` offre le funzionalità di un client SSL/TLS (`man s_client`). Il comando `s_server` offre invece le funzionalità di un semplice server SSL/TLS (`man s_server`).

I file di configurazione necessari per svolgere alcuni esercizi sono disponibili sul sito web del corso:

`05lab-materiale.zip`

1.1 Comandi accessori

Nel corso dell'esercitazione dovrete scambiarvi dati da un computer all'altro. Per farlo con SSH e il comando `scp`, abilitate il server `ssh`:

```
systemctl start ssh
```

Per copiare un file (e.g. `prova`) dalla macchina `ssh` client nella directory `root` della macchina `ssh` server, potete usare il comando:

```
scp prova root@IP_address_ssh_server:/root
```

ed inserire la password quando richiesto (se non l'avete cambiata è 'toor').

Verrà inoltre usato il server web Apache; per avviarlo usate il comando:

```
systemctl start apache2
```

Nel corso dell'esercitazione dovrete generare certificati X.509 per server e client, come visto nell'esercitazione precedente.

1.2 Comandi utili: IPsec e sicurezza a livello IP

ip xfrm

`ip xfrm` è lo strumento usato per manipolare le "security association" e le "security policy" di IPsec. include i seguenti sotto-comandi, come descritto in `man ip xfrm`:

- `state`, per gestire il database delle Security Associations (SA), usando i seguenti sotto-comandi:
  - `add | update` per aggiungere o aggiornare una SA;
  - `list` per elencare tutte le SA nel DB di SA (SAD);
  - `flush` per eliminare tutte le SA di un SAD;
- `policy`, per gestire il DB delle Security Policies (SP), usando i seguenti sotto-comandi:
  - `add | update` per aggiungere o aggiornare una SP;
  - `list` per elencare tutte le SP nel DB di SP (SPD);
  - `flush` per eliminare tutte le SP di un SPD;
- `monitor`, per mostrare informazioni di debug riguardo SA ed SP.

1.2.1 IPsec Security Associations (ip xfrm state)

Per creare delle nuove SA si può usare il comando `add`, secondo la seguente sintassi:

```
ip xfrm state add [src IP_src] [dst IP_dst] [proto type_of_protection] [spi SPI]
[protection_methods]
```

Le SA sono unidirezionali, nel senso che definiscono quale protezione deve essere applicata per i pacchetti inviati da *IP_src* a *IP_dst*. Per proteggere anche il traffico di ritorno (ossia da *IP_dst* a *IP_src*) ed ottenere quindi un canale bidirezionale, è necessario definire un'altra SA.
Il *tipo_di_protezione* è una delle opzioni messe a disposizione da IPsec, cioè AH (`ah`) o ESP (`esp`) o 'compression' (`comp`).
Il parametro *SPI* (Security Parameter Index) è un numero intero (espresso in decimale o in esadecimale con il prefisso `0x`) che serve ad indicizzare le SA presenti nel SAD al momento della loro applicazione. I valori da 0 a 255 sono riservati. E' buona norma e pratica consueta lasciare le ultime tre cifre esadecimali fisse (tre zeri), cominciare dal valore `0x1000` ed incrementare a partire dalla quarta cifra, cioè `0x2000`, `0x3000`, ecc.
Per quanto riguarda i *metodi_di_protezione* esistono quattro opzioni:

- `enc`, per specificare l'algoritmo di cifratura (es. `aes`) e la chiave segreta da usare
- `auth`, per specificare l'algoritmo di digest da usare per HMAC (es. `sha1`, `sha256`) ed il segreto da usare
- `aead`, per specificare l'algoritmo di authenticated encryption with associated data, la chiave e la lunghezza dell'integrity check value
- `comp`, per specificare l'algoritmo di compressione (es. `deflate`, `lzs`).

La lista completa degli algoritmi supportati di cifratura, autenticazione e compressione è consultabile usando il `man` del comando `ip xfrm state`.

1.2.2 IPsec Security Policies (ip xfrm policy)

Per definire a quale tipo di traffico si applica una determinata SA è necessario definire delle SP. Per farlo è possibile usare il sotto-comando `policy` che ha la seguente sintassi:

```
ip xfrm policy add [src IP_src] [dst IP_dst] [proto [PROTOCOL] dir DIR
[action ACTION] [tmpl POLICY]
```

Anche le politiche, in quanto "selettori" di SA da applicare, sono unidirezionali.
Il campo *DIR* deve indicare o il traffico di input (`in`) o il traffico di output (`out`) o pacchetti che devono essere inoltrati (`fwd`).
Il campo *PROTOCOL* rappresenta un protocolllo valido tra `tcp`, `udp`, `sctp`, `dccp`, `icmp`, `icmp-ipv6` and `gre`. In caso di protocolli di trasporto, `sport` e `dport` possono essere usate per specificare porte sorgente e destinazione, rispettivamente. In caso questo parametro non sia specificato, la SP si applica a tutti i protocolli.
Il campo *ACTION* definisce l'azione da effettuare ogni volta un pacchetto corrisponda alla policy, e può essere 'permesso' (`allow`, valore di default) or 'bloccato' (`block`).
Il campo *POLICY* include i seguenti parametri:

- `proto`: `ah` oppure `esp`.
- `mode`: indica o IPsec transport oppure tunnel mode. Se `mode` è `tunnel`, vanno specificati gli indirizzi degli end-point della SA come `src` e `dst`, usati per identificare la SA da usare. Se `mode` è `transport`, `src` e `dst` possono essere omessi.
- `level`: può essere `required` (default) o `use`. Il parametro `required` significa che una SA è richiesta ogni volta che il kernel manda un pacchetto che ricada sotto questa policy (l'operazione fallisce in caso non sia disponibile almeno una SA). `use` significa che il kernel usa an SA se disponibile, altrimenti prosegue con le normali operazioni previste in assenza di una policy.

strongSwan

Per evitare la creazione manuale delle SA, è possibile usare il protocollo IKE per crearle dinamicamente.
Il demone usato per il protocollo IKE è chiamato `strongSwan` ed è un'implementazione opensource di IPsec per Linux.
Il comando seguente serve per avviare il demone strongSwan:

```
ipsec start
```

Il comando seguente serve per riavviare il demone (per esempio dopo aver cambiato la configurazione):

```
ipsec restart
```

Il comportamento di strongSwan è principalmente controllato dal file di configurazione `/etc/ipsec.conf` ove vengono indicate le direttive utili alla negoziazione delle chiavi effettuata dal demone `charon` e per la negoziazione delle SA.
In aggiunta, è necessario usare il file `/etc/ipsec.secrets` che contiene segreti quali le chiavi private RSA o chiavi precedentemente condivise ("pre-shared").
Va usata anche la directory `/etc/ipsec.d`, la quale contiene certificati e CRL usate dal keying daemon `charon`.
Il file di configurazione di strongSwan `/etc/ipsec.conf` consiste principalmente di tre sezioni differenti:

- `config setup` definisce parametri di configurazione generali
- `conn [` *name* `]` definisce una connessione

- `ca [` *name* `]` definisce una Certification Authority

Negli esercizi proposti voi modificherete principalmente la sezione `conn [` *name* `]`. Nella sezione `conn`, le istruzioni `left` e `right` (che andranno cambiate nel file `/etc/ipsec.con` sulla base della specifica configurazione dell'host con cui state lavorando) denotano i due endpoint di una IKE SA: `left` indica il computer locale, cioè quello in cui il file `ipsec.conf` è memorizzato, mentre `right` indica il computer remoto. Come suggerimento mnemonico, potete pensare alla prima lettera delle parole (left=locale, right=remoto).
Oltre ad autenticazione e materiale per scambio di chiavi, IKE fornisce anche i mezzi per lo scambio di informazioni di configurazione e per la negoziazione di IPsec SA (chiamate CHILD_SA). Le SA definiscono quale traffico di rete deve essere reso sicuro e come possa essere cifrato ed autenticato.
Una CHILD_SA è composta da due componenti, la SA IPsec descrivente gli algoritmi e le chiavi usate per cifrare e autenticare il traffico, e le policy che definiscono quale traffico di rete usi una SA. Le policy funzionano in entrambi i modi, questo significa che in `tunnel` mode con `esp` si può valutare la policy di ingresso solo dopo la decifratura.

TLS

La libreria OpenSSL implementa due semplici client e server SSL/TLS, che possono essere usati per il test di questi protocolli.
Il comando OpenSSL usato per avviare il client è `s_client` con la sintassi:

```
openssl s_client [-connect host:port] [-state] [-showcert] [-CAfile file_cert]
[-cipher cipherlist] [-reconnect]
```

dove:

- `-connect` *host:port* specifica host e opzionalmente porta a cui connettersi (se host e porta non sono specificati, viene usato come default localhost e porta 4433);
- `-state` stampa lo stato delle sessioni TLS;
- `-showcerts` mostra la catena di certificati del server (normalmente viene visualizzato solo il certificato del server);
- `-CAfile` *file* indica il file contenente i certificati fidati da usare durante la "server authentication", e per costruire la catena di certificati del client;
- `-cipher` *cipherlist* permette di specificare la lista di ciphersuite inviata dal client nel messaggio "ClientHello" del protocollo di handshake. Per determinare quale ciphersuite usare il server dovrebbe prendere il primo algoritmo supportato presente nella lista inviata dal client. Per maggiori informazioni, vedere il comando OpenSSL `ciphers`.
- `-reconnect` permette al client di riconnettersi allo stesso server cinque volte usando lo stesso session ID.

Il comando usato per avviare il server TLS è `s_server` con la sintassi:

```
openssl s_server [-www] [-no_dhe] [-key server_pkey.pem] [-cert server_cert.pem]
[-CAfile file_cert] [-{vV}erify depth] [-cipher ciphersuite_list]
```

dove:

- `-www` invia un messaggio di status al client quando quest'ultimo prova a connettersi. Questo include diverse informazioni riguardo gli algoritmi di cifratura usati e vari parametri di sessione. L'output è in HTML, pertanto questa opzione viene normalmente usata con un browser web.

- `-no_dhe` se questa opzione è attivata non vengono creati parametri DH, disabilitando le ciphersuite che usano "ephemeral DH" (DHE).
- `-key` *server_pkey.pem* indica che il file `server_pkey.pem` contiene la chiave privata del server.
- `-cert` *server_cert.pem* indica che il file `server_cert.pem` contiene il certificato del server.
- `-CAfile` *file* indica il file contenente i certificati fidati da usare durante la client authentication e da usare per ricostruire la catena di certificati del server. Questa lista è anche usata come lista di CA accettabili passata al client quando è richiesta la "client authentication".
- `-verify` *depth* o `-Verify` *depth* indicano la profondita da usare nella verifica. Specifica la lunghezza massima della catena di certificati del client e richiesta al client nella server request. Con l'opzione `-Verify` il client è tenuto a fornire un certificato altrimenti viene causato un errore e concluso lo scambio.
- `-cipher` *ciphersuite_list* modifica la ciphersuite list del server. Quando un client inoltra una lista di ciphersuite supportate, viene scelta la prima inclusa anche nella lista del server. Poiché il client specifica l'ordine di preferenza, l'ordine della lista del server risulta irrilevante. Per ulteriori informazioni consultare il comando OpenSSL `ciphers`.

2 IPSec e la sicurezza a livello IP

2.1 Definizione di politiche e associazioni IPsec

Formate delle coppie e provate ad instaurare tra le due macchine un canale IPsec.

Per prima cosa analizzate il contenuto del man per `ip xfrm` e identificate i comandi richiesti per cancellare ed elencare il contenuto di SAD e SPD e segnatevelo, in quanto richiesto nel proseguio dell'esercitazione.

Soluzione

Le direttive `ip xfrm state flush` e `ip xfrm policy flush` servono a cancellare rispettivamente il contenuto del Security Association Database (SAD) e del Security Policy Database (SPD). Le direttive `ip xfrm state list` e `ip xfrm policy list` elencano il contenuto di SAD e SPD.

Per creare le SA e le policy tra Alice e Bob, Alice esegue le seguenti istruzioni:

```
ip xfrm state add src IP_Alice dst IP_Bob proto esp spi 0x1000 enc aes
0xaa223344556677889900aabbccddeeff

ip xfrm state add src IP_Bob dst IP_Alice proto esp spi 0x2000 enc aes
0xbb223344556677889900aabbccddeeff

ip xfrm policy add src IP_Alice dst IP_Bob dir out tmpl proto esp mode transport
level required

ip xfrm policy add src IP_Bob dst IP_Alice dir in tmpl proto esp mode transport
level required
```

Per creare le SA e le policy tra Alice e Bob, Bob esegue le seguenti istruzioni:

```
ip xfrm state add src IP_Alice dst IP_Bob proto esp spi 0x1000 enc aes
0xaa223344556677889900aabbccddeeff

ip xfrm state add src IP_Bob dst IP_Alice proto esp spi 0x2000 enc aes
0xbb223344556677889900aabbccddeeff

ip xfrm policy add src IP_Alice dst IP_Bob dir in tmpl proto esp mode transport
level required

ip xfrm policy add src IP_Bob dst IP_Alice dir out tmpl proto esp mode transport
level required
```

Rispondete ora alle seguenti domande:

- quale traffico viene protetto con IPsec?

  Soluzione

  Il payload IP (e potenzialmente parte dell'header IP con AH) in transport mode

- quale protocollo IPsec viene utilizzato? Quali proprietà di sicurezza sono fornite? Quali algoritmi crittografici sono usati?

  Soluzione

  Viene usato Encapsulating Security Payload (ESP) in transport mode. Fornisce confidenzialità e protezione dagli attacchi reply. E' usato l'algoritmo AES-CBC per la crittografia.
  Fornisce anche integrità e autenticazione se viene definito un meccanismo di autenticazione (come proposto più avanti al punto 2.2).

- quale ruolo giocano rispettivamente il Security Association Database (SAD) e il Security Policy Database (SPD) nell'architettura IPsec?

  Soluzione

  Nel Security Association Database ci sono le SA: algoritmi e chiavi definiti da Alice e Bob devono essere gli stessi. Nel Security Policy Database ci sono le SP: descrivono quando usare quali SA. Per Alice e Bob il campo `dir` è speculare (in/out)

- quali direttive saranno considerate quando Alice invia un nuovo pacchetto IP?

  Soluzione

  La SP relativa a"out" quando deve inviare un pacchetto, più la corrispondente SA

- quali direttive saranno considerate quando Alice riceve un nuovo pacchetto IP?

  Soluzione

  La SP relativa a "in" quando riceve un pacchetto, più la corrispondente SA

- perché sono necessarie due SA?

  Soluzione

  Perché la SA IPsec è unidirezionale

NOTA

E' possibile definire anche policy di tipo "forward". La policy di forward descrive quali impostazioni usare per i pacchetti che la macchina dovrebbe inoltrare (qualora fosse configurata per questo scopo), cioè spiega come trattare pacchetti che non siano né destinati alla macchina locale né forgiati dalla macchina locale.

Eseguite i comandi con gli indirizzi IP delle vostre macchine (corrispondenti ad Alice e Bob). Attivate la SA e la SP e verificate che siano state attivate correttamente controllando il contenuto di SAD e SPD.

Ora inviate un ping alla macchina partner, analizzate il traffico generato (ad esempio con `wireshark`), e rispondete alle seguenti domande:

- qual è lo scopo del campo SPI presente nei pacchetti scambiati?

Soluzione

Identificare la SA applicata

- qual è lo scopo del campo Sequence Number?

Soluzione

Impedire gli attacchi di tipo replay

2.2 Modifica di politiche e associazioni IPsec

Descrivete le modifiche alle SA e SP definite in precedenza per usare i seguenti meccanismi di sicurezza:

1. ESP con AES-128-CBC e HMAC-SHA1;

Soluzione

Occorre aggiungere nelle precedenti definizioni delle SA i parametri `auth hmac(sha1)` [*chiave_da_160_bit*].

NOTA

Il terminale interpreta le parentesi tonde come caratteri speciali, quindi l'algoritmo va scritto come `hmac\(sha1\)` oppure `"hmac(sha1)"`

Ad esempio per Alice:

```
ip xfrm state add src IP_Alice dst IP_Bob proto esp spi 0x1000
enc aes 0xaa223344556677889900aabbccddeeff auth "hmac(sha1)"
0xaa1222333444555666777888999000aaabbbcccd
```

```
ip xfrm state add src IP_Bob dst IP_Alice proto esp spi 0x2000
enc aes 0xbb223344556677889900aabbccddeeff auth "hmac(sha1)"
0xbb1222333444555666777888999000aaabbbcccd
```

```
ip xfrm policy add src IP_Alice dst IP_Bob dir out tmpl proto esp mode
transport level required
```

```
ip xfrm policy add src IP_Bob dst IP_Alice dir in tmpl proto esp mode
transport level required
```

Ad esempio per Bob:

```
ip xfrm state add src IP_Alice dst IP_Bob proto esp spi 0x1000
enc aes 0xaa22334455667788990aabbccddeeff auth "hmac(sha1)"
0xaa122233344455566677888999000aaabbbcccd
```

```
ip xfrm state add src IP_Bob dst IP_Alice proto esp spi 0x2000
enc aes 0xbb22334455667788990aabbccddeeff auth "hmac(sha1)"
0xbb122233344455566677888999000aaabbbcccd
```

```
ip xfrm policy add src IP_Bob dst IP_Alice dir out tmpl proto esp mode
transport level required
```

```
ip xfrm policy add src IP_Alice dst IP_Bob dir in tmpl proto esp mode
transport level required
```

2. AH con HMAC-SHA1;

Soluzione

Nella definizione della SA bisogna sostituire `esp` con `ah` e cambiare l'algoritmo. Nella definizione della SP bisogna sostituire `esp` con `ah`.
Ad esempio per Alice:

```
ip xfrm state add src IP_Alice dst IP_Bob proto ah spi 0x1000 auth
"hmac(sha1)" 0xaa122233344455566677888999000aaabbbcccd
```

```
ip xfrm state add src IP_Bob dst IP_Alice proto ah spi 0x2000 auth
"hmac(sha1)" 0xbb122233344455566677888999000aaabbbcccd
```

```
ip xfrm policy add src IP_Alice dst IP_Bob dir out tmpl proto ah mode
transport level required
```

```
ip xfrm policy add src IP_Bob dst IP_Alice dir in tmpl proto ah mode
transport level required
```

Ad esempio per Bob:

```
ip xfrm state add src IP_Alice dst IP_Bob proto ah spi 0x1000 auth
"hmac(sha1)" 0xaa122233344455566677888999000aaabbbcccd
```

```
ip xfrm state add src IP_Bob dst IP_Alice proto ah spi 0x2000 auth
"hmac(sha1)" 0xbb122233344455566677888999000aaabbbcccd
```

```
ip xfrm policy add src IP_Bob dst IP_Alice dir out tmpl proto ah mode
transport level required
```

```
ip xfrm policy add src IP_Alice dst IP_Bob dir in tmpl proto ah mode
transport level required
```

3. ESP con AES-128-CBC e AH con HMAC-SHA1.

NOTA

La modalità ESP+AH è stata dichiarata NOT RECOMMENDED dall'RFC-8221, Sezione 4 (https://www.ietf.org/rfc/rfc8221.html#section-4). E' da preferirsi la soluzione mostrata al successivo punto 4 che introduce meno overhead

Soluzione

Alle due SA con `esp` ne vanno aggiunte altre due con `ah`. Le due SP vanno modificate aggiungendo entrambi i meccanismi `esp` ed `ah`.
Ad esempio per Alice:

```
ip xfrm state add src IP_Alice dst IP_Bob proto esp spi 0x1000 enc aes
0xaa223344556677889900aabbccddeeff
```

```
ip xfrm state add src IP_Bob dst IP_Alice proto esp spi 0x2000 enc aes
0xbb223344556677889900aabbccddeeff
```

```
ip xfrm state add src IP_Alice dst IP_Bob proto ah spi 0x1000 auth
"hmac(sha1)" 0xaa12223334445556667778889990000aaabbbcccd
```

```
ip xfrm state add src IP_Bob dst IP_Alice proto ah spi 0x2000 auth
"hmac(sha1)" 0xbb12223334445556667778889990000aaabbbcccd
```

```
ip xfrm policy add src IP_Alice dst IP_Bob dir out tmpl proto esp mode
transport level required tmpl proto ah mode transport level required
```

```
ip xfrm policy add src IP_Bob dst IP_Alice dir in tmpl proto esp mode
transport level required tmpl proto ah mode transport level required
```

Ad esempio per Bob:

```
ip xfrm state add src IP_Alice dst IP_Bob proto esp spi 0x1000 enc aes
0xaa223344556677889900aabbccddeeff
```

```
ip xfrm state add src IP_Bob dst IP_Alice proto esp spi 0x2000 enc aes
0xbb223344556677889900aabbccddeeff
```

```
ip xfrm state add src IP_Alice dst IP_Bob proto ah spi 0x1000 auth
"hmac(sha1)" 0xaa12223334445556667778889990000aaabbbcccd
```

```
ip xfrm state add src IP_Bob dst IP_Alice proto ah spi 0x2000 auth
"hmac(sha1)" 0xbb12223334445556667778889990000aaabbbcccd
```

```
ip xfrm policy add src IP_Bob dst IP_Alice dir out tmpl proto esp mode
transport level required tmpl proto ah mode transport level required
```

```
ip xfrm policy add src IP_Alice dst IP_Bob dir in tmpl proto esp mode
transport level required tmpl proto ah mode transport level required
```

4. ESP con AES-128-GCM (RFC-4106) AEAD (Authenticated Encryption with Associated Data).

NOTA

AEAD, oltre alla chiave, richiede la lunghezza dell'Integrity Check Value (ICV) in bit.
L'RFC corrispondente (https://tools.ietf.org/html/rfc4106) specifica che AEAD accetta obbligatoriamente ICV di lunghezza 16 byte e facoltativamente di lunghezza 8 o 12 byte (Section

6).
Il materiale crittografico richiesto per AES-GCM-ESP a 128 bit ha lunghezza complessiva di 20 byte: i primi 16 sono la chiave a 128 bit, i rimanenti 4 sono il 'salt' del nonce (Section 8.1)

Soluzione

Occorre modificare le SA cambiando il metodo di protezione da `enc` ad `aead`. Bisogna cambiare l'algoritmo da `aes` a `"rfc4106(gcm(aes))"`, usare una chiave da 20 byte e aggiungere la lunghezza dell'ICV.
Ad esempio per Alice:

```
ip xfrm state add src IP_Alice dst IP_Bob proto esp spi 0x1000 aead
"rfc4106(gcm(aes))" 0xaa223344556677889900aabbccddeeffaabbccdd 128
```

```
ip xfrm state add src IP_Bob dst IP_Alice proto esp spi 0x2000 aead
"rfc4106(gcm(aes))" 0xbb223344556677889900aabbccddeeffaabbccdd 128
```

```
ip xfrm policy add src IP_Alice dst IP_Bob dir out tmpl proto esp mode
transport level required
```

```
ip xfrm policy add src IP_Bob dst IP_Alice dir in tmpl proto esp mode
transport level required
```

Ad esempio per Bob:

```
ip xfrm state add src IP_Alice dst IP_Bob proto esp spi 0x1000 aead
"rfc4106(gcm(aes))" 0xaa223344556677889900aabbccddeeffaabbccdd 128
```

```
ip xfrm state add src IP_Bob dst IP_Alice proto esp spi 0x2000 aead
"rfc4106(gcm(aes))" 0xbb223344556677889900aabbccddeeffaabbccdd 128
```

```
ip xfrm policy add src IP_Bob dst IP_Alice dir out tmpl proto esp mode
transport level required
```

```
ip xfrm policy add src IP_Alice dst IP_Bob dir in tmpl proto esp mode
transport level required
```

NOTA

Usate `man ip xfrm` per elencare tutti gli algoritmi crittografici supportati. Ricordate che le parentesi "(" e ")" devono essere precedute dai caratteri di escape di black-slash.
Ricordate anche che le chiavi devono essere di lunghezza esatta.

Suggerimento: quante SA e quante SP dovete definire per realizzare la soluzione 3?

Soluzione

Bastano sempre due SP, una in ingresso ed una in uscita; Servono invece quattro SA, due per il canale ESP e due per il canale AH

Per ognuna delle configurazioni descritta sopra, attivate le SA modificate, inviate un ping alla macchina partner, analizzate i pacchetti scambiati (ad esempio con `wireshark`), e rispondete alle seguenti domande:

- che differenza c'è tra le configurazioni 1 e 3 in termini di struttura dei pacchetti risultanti? E in termini di protezione fornita?

Soluzione

In modalità ESP c'è una sola intestazione. In modalità ESP+AH ce ne sono due, quella per AH e quella per ESP, e questo comporta un maggiore overhead.
La modalità ESP (in transport mode) non protegge l'intestazione IP, mentre AH in parte si.

- quando la configurazione 2 può risultare opportuna?

Soluzione

La configurazione 2 non cifra i dati inviati. Quindi è opportuna quando è necessario proteggere l'integrità dei dati ma non la confidenzialità

- quali sono i vantaggi della configurazione 4 rispetto alle altre?

Soluzione

La configurazione 4 protegge sia l'integrità del messaggio, sia la segretezza del payload attraverso un singolo algoritmo (ed una singola chiave) rispetto agli schemi basati sull'uso congiunto di ESP ed AH, che non sono più raccomandati. Data la sua efficienza, consente alle connessioni IPsec di supportare dispositivi di rete con capacità 10-gigabit e 40-gigabit (da RFC).

NOTA

Con un algoritmo AEAD si può proteggere parzialmente anche l'integrità dell'intestazione IP (seppur in misura meno estesa rispetto ad AH) copiandone alcuni campi negli Associated Data. Ad esempio con l'AES-128-GCM (RFC-4106) visto al punto 4 si copiano negli Associated Data i campi SPI ed (Extendend) Sequence Number
https://tools.ietf.org/html/rfc4106#section-5

Scegliete una delle precedenti configurazioni e modificate le SP affinché non vengano protetti tutti i protocolli, ma solamente il TCP. Verificate quindi che i pacchetti ping non vengono protetti, mentre lo è il traffico TCP (ad esempio quello HTTP).

Soluzione

Occorre inserire la direttiva `proto tcp` dopo la definizione degli indirizzi IP in ciascuna SP. Ad esempio:

```
ip xfrm policy add src IP_Alice dst IP_Bob proto tcp dir out tmpl proto esp
mode transport level required
```

Per riassumere, sapreste indicare quali informazioni sono contenute in SAD e quali in SPD?

Soluzione

L'SPD contiene le SP che definiscono il tipo di traffico e stabiliscono se questo deve essere protetto o no con IPsec, se deve essere bloccato, definisce la modalità tunnel o traporto, gli algoritmi in ordine di preferenza, ecc.
L'SAD contiene le SA che definiscono i tipi di protezione (ah, esp) e metodi di protezione (enc, auth, aead, ecc.) e la relative chiavi, IV, ecc.
Per maggiori dettagli consultare l'RFC4301 https://tools.ietf.org/html/rfc4301

2.3 Misure di prestazione

Innanzitutto occorre effettuare la misura dei tempi di trasferimento col canale in condizioni normali, ossia senza IPsec.

Per prima cosa, svuotate SAD e SPD:

```
ip xfrm state flush
```

```
ip xfrm policy flush
```

Formate dunque delle coppie, diciamo Alice e Bob, e procedente come segue:

1. Alice crea la nuova directory `/var/www/html/ipsec/` e, al suo interno, una serie di file di dimensione 10 kB, 100 kB, 1 MB, 10 MB, 100 MB, nominandoli rispettivamente `10k`, `100k`, `1M`, `10M` e `100M`. Per creare file di una determinata lunghezza si può usare il seguente comando:

   ```
   openssl rand -out nome_file dimensione_in_byte
   ```

2. Alice attiva quindi il server Apache

3. Bob scarica i file appena creati da Alice e prende nota dei tempi richiesti dal trasferimento; usate il client HTTP a riga di comando `curl`:

   ```
   curl --trace-time -v -o /dev/null http://IP_Alice/ipsec/nome_file
   ```

4. riportate nella tabella 1 il tempo di trasferimento e la relativa velocità per i diversi file

   NOTA

   Compatibilmente con il tempo a disposizione, si consiglia di ripetere il trasferimento più volte e mediare i risultati

Ora eseguite gli stessi test di prestazione per le diverse configurazioni IPsec provate negli esercizi precedenti; riportate i risultati in tabella 1 e confrontateli con quelli precedenti.

Soluzione

Per quanto riguarda IPsec le prestazioni migliori si hanno in modalità ESP con AEAD (punto 4), seguito da ESP con solo cifratura (come definito nella sezione 2.1). Segue la modalità AH (punto 2) mentre in coda troviamo sostanzialmente sovrapposte la modalità ESP con cifratura e HMAC (punto 1) e la modalità ESP+AH (punto 3).

2.4 Il protocollo IKE

Se non si vogliono o non è conveniente creare manualmente le SA, si può utilizzare il protocollo IKE che permette di crearle dinamicamente. Il demone che gestisce il protocollo IKE è strongSwan. Fate riferimento alla precedente sezione 1.2 inerente strongSwan per dettagli sui comandi.

2.4.1 strongSwan e pre-shared-keys

Formate due gruppi di host, Alice e Bob, quindi, dall'archivio contenente i file di supporto del laboratorio estraete nelle rispettive macchine il contenuto della cartella `/strongswan/IKE_with_psk(Ali-Bob)`.

Alice effettua i seguenti passi:

- rimuove le entry nel database SAD con il comando `ip xfrm state flush`; quindi rimuove le entry nel database SPD con il comando `ip xfrm policy flush` (in questo esercizio, SA e SP verranno create dal demone strongSwan);

	10 kB	100 kB	1 MB	10 MB	100 MB
No IPsec/TLS					
time [s]					
speed [kB/s]					
ESP transport AES-128-CBC					
time [s]					
speed [kB/s]					
ESP transport AES-128-CBC HMAC-SHA1					
time [s]					
speed [kB/s]					
AH transport HMAC-SHA1					
time [s]					
speed [kB/s]					
ESP AES-128-CBC + AH HMAC-SHA1					
time [s]					
speed [kB/s]					
ESP AES-128-GCM					
time [s]					
speed [kB/s]					
TLS RSA with AES-128-CBC and SHA1, solo server auth					
time [s]					
speed [kB/s]					
TLS RSA with AES-128-CBC and SHA1 solo client auth					
time [s]					
speed [kB/s]					

Tabella 1: Misure di prestazione.

- rimpiazza il contenuto del file `/etc/ipsec.conf` con quello nel file `ipsec.conf_psk_ali`;
- nel file `/etc/ipsec.conf` modifica gli indirizzi IP con quelli di Alice e Bob; Per esempio, nel file fornito, 192.168.27.128 è l'indirizzo di Alice, mentre 192.168.27.132 è l'indirizzo IP di Bob.

> NOTA
>
> prestate attenzione anche alle parti `leftsubnet` e `rightsubnet`, possono necessitare di cambiamenti in accordo con la configurazione di rete.

- rimpiazza il contenuto del file `/etc/ipsec.secrets` con quello nel file `ipsec.secrets_psk_ali`;
- nel file `/etc/ipsec.secrets` modifica gli indirizzi IP con quelli di Alice e Bob.

Bob effettua i seguenti passi:

- rimuove le entry nel database SAD con il comando `ip xfrm state flush`; quindi rimuove le entry nel database SPD con il comando `ip xfrm policy flush`; (in questo esercizio, SA e SP verranno create dal demone strongSwan);
- rimpiazza il contenuto del file `/etc/ipsec.conf` con quello nel file `ipsec.conf_psk_bob`;
- nel file `/etc/ipsec.conf` modifica gli indirizzi IP con quelli di Alice e Bob, prestando attenzione all'ordine; nel file fornito, 192.168.27.128 è l'indirizzo di Alice, mentre 192.168.27.132 è l'indirizzo IP di Bob.

NOTA

Prestate attenzione anche alle parti `leftsubnet` e `rightsubnet`, possono necessitare di cambiamenti in accordo con la configurazione di rete.

- rimpiazza il contenuto del file `/etc/ipsec.secrets` con quello nel file `ipsec.secrets_psk_bob`.
- nel file `/etc/ipsec.secrets` modifica gli indirizzi IP con quelli di Alice e Bob, prestando attenzione all'ordine; nel file fornito 192.168.27.128 è l'indirizzo di Alice, mentre 192.168.27.132 è l'indirizzo IP di Bob.

Alice attiva strongSwan con il comando:

```
ipsec start
```

Bob attiva strongSwan con il comando:

```
ipsec start
```

Alice attiva la connessione configurata in `/etc/ipsec.conf` con il comando:

```
ipsec up host-host
```

Analizzate con il comando `ip xfrm state list` le SA IPsec create dal demone IKE sulle macchine di Alice e Bob, e rispondere alle seguenti domande:

Quante SA sono state negoziate? Si suggerisce man `ipsec.conf` per approfondimenti.

Soluzione

Due, una per ogni direzione

Quale tipo di security header e di modo viene usato nelle SA? ESP o AH? Quali algoritmi sono usate? Qual è la lunghezza della chiave usata per ogni algoritmo?

Soluzione

Viene usato `esp` in `tunnel mode`. Per la cifratura si usa `aes-cbc` (con chiave a 128 bit) mentre per integrità e autenticazione si ricorre a `hmac-sha256` con chiave a 256 bit

In quale fase del protocollo IKE è usata la PSK?

Soluzione

La prima, quella per la creazione della SA IKE per stabilire un canale sicuro in cui durante la fase 2 verrá negoziata la SA IPsec.

Analizzate con il comando `ip xfrm policy list` le policy IPsec create dal demone IKE sulle macchine di Alice e Bob.

Ale genera traffico verso la macchina di Bob (per esempio, icmp e http) e lo analizza (per esempio con `wireshark`).

NOTA

se volete osservare diverse volte la negoziazione IKE, è necessario svuotare SAD e SPD, oppure riavviare `strongSwan` ed attivare la connessione (con `ipsec up host-host`).

ATTENZIONE

prima di passare agli esercizi successivi, ricordatevi di disattivare strongSwan e svuotare SAD e SPD

3 Il protocollo TLS

3.1 Instaurare un canale TLS

Provate a collegarvi con il browser a un server TLS, ad esempio `https://mail.polito.it`.

Analizzate il certificato X.509 presentato dal server: quali campi qualificano l'identità del server?

Soluzione

Il campo Subject, in particolare il Common Name (CN):

```
Subject:  C = IT, L = Torino, O = Politecnico di Torino, CN =
mail.polito.it
```

La URL del server è specificata nell'estensione Subject Alternative Name:

```
X509v3 Subject Alternative Name:
DNS:mail.polito.it
```

Qual è la catena di certificazione?

Soluzione

USERTrust RSA Certification Authority → GEANT → mail.polito.it

Quali algoritmi sono usati per la protezione effettiva dei dati?

Soluzione

Ce lo dice il browser (testato con Firefox 115) e non il certificato perché la scelta è frutto della negoziazione TLS. Nel caso in esame è stata negoziata queste ciphersuite:
`https://ciphersuite.info/cs/TLS_AES_256_GCM_SHA384/`

Ora provate a collegarvi allo stesso server dell'esercizio precedente usando `s_client`:

```
openssl s_client -connect mail.polito.it:443 -state -showcerts -verify 1
```

Pensate di aver specificato tutti i dati necessari a stabilire una connessione TLS corretta? Eppure la connessione è andata a buon fine. Perché?

Soluzione

Manca il certificato della CA ma la connessione va a buon fine perché TLS ha la lista delle rootCA trusted

Riuscite a identificare la sequenza di operazioni seguite da `s_client`? Più precisamente: a cosa si riferiscono le operazioni `read` e `write` elencate all'inizio della sequenza?

Soluzione

Rappresentano la negoziazione (handshake) TLS tra client e server. In questa fase avviene l'autenticazione (obbligatoria per il server, facoltativa per il client) e si stabiliscono gli algoritmi crittografici

(ciphersuite) da usare per la connessione

A chi appartengono i certificati riportati nella sequenza?

Soluzione

I certificati riportati nella sequenza sono quello del server (mail.polito.it) e quello della CA intermedia (GEANT)

Cosa sono i parametri riportati nella sezione chiamata `SSL-Session`?

Soluzione

I parametri riportati sono il risultato della negoziazione TLS. A questa sessione è associato un identificativo che può essere presentato dal client al server quando apre una nuova connessione per evitare di dover rifare tutta la fase di negoziazione. Il campo timeout indica per quanto tempo può essere riutilizzata questa sessione prima che venga dimenticata dal server.

Provate a collegarvi a `https://mail.polito.it` forzando `s_client` a usare NULL come suite crittografica e esaminate il risultato. Scrivete comando e risultato:

NOTA

Il supporto alla ciphersuite NULL é stato rimosso in TLS 1.3, é quindi necessario forzare la connessione a TLS 1.2 con il flag `-tls1_2`.

Soluzione

Il comando è:

```
openssl s_client -connect mail.polito.it:443 -state -showcerts -verify 1
-cipher NULL -tls1_2
```

Il risultato è che la connessione fallisce.

3.2 Configurazione di un server TLS

Provate ora a configurare un server TLS. Di cosa avete bisogno?

Potete generare un certificato per il server nel seguente modo:

1. create una CA di prova come fatto nell'esercitazione precedente:

   ```
   /usr/lib/ssl/misc/CA.pl -newca
   ```

2. create una richiesta di certificato per il server:

   ```
   openssl req -new -keyout server_pkey.pem -out server_creq.pem
   ```

3. emettete il nuovo certificato:

   ```
   openssl ca -in server_creq.pem -out server_cert.pem
   ```

Attivate `s_server` con il seguente comando:

```
openssl s_server -www -no_dhe -key server_pkey.pem -cert server_cert.pem
```

Il server si metterà in ascolto sulla porta `4433`. Provate a collegarvi con `s_client` e esaminate il risultato:

```
openssl s_client -connect 127.0.0.1:4433 -state -showcerts -CAfile
demoCA/cacert.pem
```

Provate a connettervi con il browser (`https://127.0.0.1:4433`): con il parametro `-www`, `s_server` ritorna al client una pagina contenente una serie di utili informazioni sulla sessione appena creata.

A cosa serve il parametro `-no_dhe` usato finora? Cosa sono e a cosa possono servire i meccanismi effimeri per lo scambio chiavi in TLS?

Soluzione

Disabilita lo scambio Diffie-Hellman effimero. I meccanismi effimeri sono meccanismo in cui viene generata una one-time key tramite il protocollo Diffie-Hellman. Possono servire per garantire la "Perfect Forward Secrecy", la chiave originale viene usata solo per firmare i parametri Diffie-Hellman

3.3 Uso del Session ID

NOTA

Per questo punto dell'esercitazione é necessario utilizzare TLS 1.2, in quanto il SessionID é considerato obsoleto in TLS 1.3 (RFC8446) e sostituito dalla resumption tramite PSK.
Di conseguenza é necessario avviare il server forzandolo ad utilizzare la precedente versione di TLS. Ció puó essere ottenuto aggiungendo il flag `-tls1_2`:

```
openssl s_server -www -no_dhe -key server_pkey.pem -cert server_cert.pem
-tls1_2
```

A cosa serve il "Session ID" in TLS?

Lanciate il seguente comando che istruisce `s_client` ad effettuare alcune connessioni consecutive sfruttando il Session ID:

```
openssl s_client -connect 127.0.0.1:4433 -state -CAfile demoCA/cacert.pem
-reconnect
```

Quali parametri della sessione TLS rimangono invariati nelle connessioni successive e quali invece cambiano? (verificate)

Soluzione

Il SessionID veniva utilizzato per avviare nuove connessioni utilizzando una chiave e una suite criptografica negoziati in precedenza ed identificati dal suddetto campo. Rimangono quindi invariati il protocollo, la ciphersuite, il session ID e la Master Key. Cambiano le altre chiavi derivate e i numeri casuali.

3.4 Autenticazione del client tramite TLS

Provate ora a configurare un server TLS con client authentication. Di cosa avete bisogno?

Potete generare un certificato per il client nel seguente modo:

1. create una richiesta di certificato per il client:

   ```
   openssl req -new -keyout client_pkey.pem -out client_creq.pem
   ```

2. emettete il nuovo certificato:

   ```
   openssl ca -in client_creq.pem -out client_cert.pem
   ```

Configurate ora `s_server` in modo da richiedere l'autenticazione del client tramite il protocollo TLS:

```
openssl s_server -www -no_dhe -key server_pkey.pem -cert server_cert.pem -CAfile
demoCA/cacert.pem -Verify 1
```

Scrivete il comando `s_client` e i dati aggiuntivi necessari a completare la connessione:

Soluzione

```
openssl s_client -connect 127.0.0.1:4433 -state -showcerts -CAfile
demoCA/cacert.pem -cert client_cert.pem -key client_pkey.pem
```

Riuscite a identificare i passi che sono cambiati all'interno della sequenza di handshake?

Soluzione

Il server ora chiede un certificato al client per autenticarlo ed il client glielo manda.

Perché il server invia ora anche il certificato della CA?

Soluzione

Perché il server, quando chiede un certificato al client, gli manda la lista delle CA accettate.

Alcuni server (ad esempio Microsoft IIS) mostrano il seguente comportamento quando viene abilitata la client authentication: all'instaurazione di un nuovo canale non inviano nessuna richiesta di certificato lato client; una volta concluso l'handshake con sola autenticazione lato server, iniziano un secondo handshake in cui è in effetti richiesta l'autenticazione del client. Quali sono i vantaggi di una simile soluzione?

Soluzione

Protegge l'identità del client (privacy) e fa si che il suo certificato sia trasmesso su un canale sicuro.

3.5 Attacco MITM a TLS

Riferitevi all'esercizio sull'attacco di man-in-the-middle visto nella prima esercitazione e lavorate in gruppi di tre: Alice, Bob e Chuck. Alice farà le veci del server web, Bob del client e Chuck sarà l'attaccante.

Riuscite a strutturare un attacco, in modo che Chuck possa sniffare il traffico tra Alice e Bob, pur trattandosi di traffico su una connessione TLS?

Provate ora a realizzare l'attacco con `ettercap`.

1. Chuck copia il file fornito `etter.conf` nel file `/etc/ettercap/etter.conf`.
2. Chuck lancia l'attacco MITM, con il comando:

   ```
   ettercap -T -M arp /IP_Alice// /IP_Bob//
   ```

3. Bob si connette con un browser al server di Alice

Riuscite a dedurre come funziona l'attacco di ettercap?

Soluzione

Crea un certificato autofirmato fasullo

Suggerimento: salvate il certificato fornito dal server e confrontatelo con quello del server web di Alice configurato nell'esercizio 3.2.

3.6 Misure di prestazione

Effettuate ora la misura dei tempi di trasferimento del canale quando protetto via TLS. Fate nuovamente riferimento alla tabella 1, formate delle coppie e procedente come segue.

1. Alice attiva il server Apache con protezione TLS lato server:
   - usa il comando `a2enmod ssl` per abilitare il modulo SSL di Apache;
   - usa il comando `a2ensite default-ssl` per abilitare il sito SSL di Apache;
   - copia i file `server_cert.pem`, `server_pkey.pem` e `demoCA/cacert.pem` nella directory di Apache per la configuazione SSL (`/etc/ssl/certs` per `server_cert.pem` e `demoCA/cacert.pem`; `/etc/ssl/private` per `server_pkey.pem`);
   - sostituisce il file `/etc/apache2/sites-enabled/default-ssl.conf` con il file fornito `apache2-ssl`;
   - per impostare l'uso di RSA con AES, sostituisce nel file `/etc/apache2/mods-available/ssl.conf` la direttiva `SSLCipherSuite HIGH:MEDIUM:!ADH:!MD5` con `SSLCipherSuite AES:SHA1:!ADH:!MD5`;
   - riavvia il server Apache, usando il noto comando:

     ```
     systemctl restart apache2
     ```

   - a scopo di verifica, prova a connettersi al suo stesso Apache server con il comando:

     ```
     openssl s_client -connect 127.0.0.1:443
     ```

2. Bob ottiene da Alice il file `cacert.pem` e il common name (CN) specificato nel file `server_cert.pem`, che indichiamo come *CN_Alice*, quindi aggiunge nel file `/etc/hosts` la riga:

   ```
   IP_Alice  CN_Alice
   ```

3. Bob scarica i file `10k`, `100k`, `1M`, `10M` e `100M` con il seguente comando:

   ```
   wget --ca-certificate=cacert.pem https://CN_Alice/ipsec/<nome-file>
   ```

   e riporta i tempi ottenuti in tabella 1;

4. Alice attiva ora l'autenticazione del client via TLS, modificando nel file `/etc/apache2/sites-enabled/default-ssl.conf` la direttiva

   ```
   SSLVerifyClient none
   ```

   con la direttiva

   ```
   SSLVerifyClient require
   ```

   e riavvia il server Apache;

5. Bob si fa dare da Alice i file `client_pkey.pem` e `client_cert.pem` (il file `cacert.pem` lo ha già ottenuto al passo 2), quindi decifra la chiave del client con il seguente comando:

   ```
   openssl rsa -in client_pkey.pem > client_decrypted_pkey.pem
   ```

6. Bob scarica i file `10k`, `100k`, `1M`, `10M` e `100M` con il seguente comando:

   ```
   wget --certificate=client_cert.pem --private-key=client_decrypted_pkey.pem
   --ca-certificate=cacert.pem https://CN_Alice/ipsec/<nome-file>
   ```

   e riporta i tempi in tabella 1.

4 Approfondimenti ed ulteriori esercizi (opzionali)

Esercizi aggiuntivi su IPsec

4.1 strongSwan e certificati X.509

In questo esercizio vedrete come configurare il demone IKE per sostituire l'autenticazione basata su pre-shared-key durante la fase 1 di IKE con l'autenticazione basata su certificati ed algoritmi a chiave pubblica.

NOTA

Nel caso non si utilizzi la VM Torsec Kali (quella del vLAIB o quella fornita tramite file ova) potrebbe essere necessario installare il package `libstrongswan-extra-plugins` per usare strongSwan con i certificati, ad esempio con il comando:

```
apt-get install libstrongswan-extra-plugins
```

Scaricate i file presenti nelle directory `Ali_machine` e `Bob_machine` che potete trovare in `strongswan/IKE_with_cert(Ali-Bob)`.
Formate gruppi di due persone, Alice e Bob.
Alice effettua i seguenti passi:

- rimuove le entry nel database SAD con il comando `ip xfrm state flush`; quindi rimuove le entry nel database SPD con il comando `ip xfrm policy flush` (in questo esercizio, SA e SP verranno create dal demone strongSwan);
- rimpiazza il contenuto del file `/etc/ipsec.conf` con quello nel file `ipsec.conf_cert_ali`;
- nel file `/etc/ipsec.conf` modifica gli indirizzi IP con quelli di Alice e Bob; Per esempio, nel file fornito, 192.168.27.128 è l'indirizzo di Alice, mentre 192.168.27.132 è l'indirizzo IP di Bob.

  NOTA

  Prestate attenzione anche alle parti `leftsubnet` e `rightsubnet`, possono necessitare di cambiamenti in accordo con la configurazione di rete.

- rimpiazza il contenuto del file `/etc/ipsec.secrets` con quello nel file `ipsec.secrets_conf_ali`;
- nel file `/etc/ipsec.secrets` modifica gli indirizzi IP con quelli di Alice e Bob;
- genera il certificato X.509 (e la chiave corrispondente) per Alice. Nella generazione delle richiesta, usare il valore "IT" per country name, "Polito" per Organization Name e "Alice" per Common Name (prestate attenzione perché il CN di Alice verrà configurato nel file `ipsec.conf_cert_bob` di Bob). Usate la password "ciao" per proteggere la chiave di Alice (prestate attenzione perché questa password viene configurata nel file `ipsec.secrets_conf_ali`). Per questo scopo potete usare i comandi:

  ```
  openssl req -newkey rsa:1024 -keyout AliKey.pem -out AliReq.pem
  ```

  ```
  /usr/lib/ssl/misc/CA.pl -newca
  ```

  ```
  openssl ca -in AliReq.pem -days 730 -out AliCert.pem -notext
  ```

  ```
  openssl rsa -in AliKey.pem -outform DER -out AliKey.der
  ```

- copia il file `AliCert.pem` nella directory `/etc/ipsec.d/certs`;
- copia il file `AliKey.der` nella directory `/etc/ipsec.d/private`;
- copia il file `cacert.pem` nel file `cacertAlice.pem` e manda il file `cacertAlice.pem` a Bob. Bob crea una demoCA sulla sua macchina (come descritto successivamente) e manda il file `cacertBob.pem` ad Alice.
- Alice copia il file `cacertBob.pem` nella directory `/etc/ipsec.d/cacerts`;
- Alice avvia strongSwan col comando:

  ```
  ipsec start
  ```

- controlla che strongSwan sia in grado di trovare la chiave privata corrispondente al certificato di Alice. A questo scopo, esegue il comando:

  ```
  ipsec listcerts
  ```

  e controlla che il certificato di Alice sia elencato e contenga la linea:

  ```
  pubkey:  RSA 1024 bits, has private key
  ```

- controlla che strongSwan sia in grado di trovare il certificato della CA di Bob. A questo scopo, esegue il comando

  ```
  ipsec listcacerts
  ```

Bob effettua i seguenti passi:

- rimuove le entry nel database SAD con il comando `ip xfrm state flush`; quindi rimuove le entry nel database SPD con il comando `ip xfrm policy flush` (in questo esercizio, SA e SP verranno create dal demone strongSwan);
- rimpiazza il contenuto del file `/etc/ipsec.conf` con quello nel file `ipsec.conf_cert_bob`;
- nel file `/etc/ipsec.conf` modifica gli indirizzi IP con quelli di Alice e Bob, prestando attenzione all'ordine; nel file fornito, 192.168.27.128 è l'indirizzo di Alice, mentre 192.168.27.132 è l'indirizzo IP di Bob.

  NOTA

  Prestate attenzione anche alle parti `leftsubnet` e `rightsubnet`, possono necessitare di cambiamenti in accordo con la configurazione di rete.

- rimpiazza il contenuto del file `/etc/ipsec.secrets` con quello nel file `ipsec.secrets_cert_bob`;
- nel file `/etc/ipsec.secrets` modifica gli indirizzi IP con quelli di Alice e Bob, prestando attenzione all'ordine; nel file fornito 192.168.27.128 è l'indirizzo di Alice, mentre 192.168.27.132 è l'indirizzo IP di Bob;
- genera il certificato (e la chiave corrispondente) per Bob. Nella generazione delle richiesta, usare il valore "IT" per country name, "Polito" per Organization Name e "Bob" per Common Name (prestate attenzione perché il CN di Bob verrà configurato nel file `ipsec.conf_cert_alice` di Alice. Usate la password "ciao" per proteggere la chiave privata di Bob (prestate attenzione perché questa password viene configurata nel file `ipsec.secrets_cert_bob`). Per questo scopo potete usare i comandi:

  ```
  openssl req -newkey rsa:1024 -keyout BobKey.pem -out BobReq.pem
  ```

```
/usr/lib/ssl/misc/CA.pl -newca
```

```
openssl ca -in BobReq.pem -days 730 -out BobCert.pem -notext
```

```
openssl rsa -in BobKey.pem -outform DER -out BobKey.der
```

- copia il file `BobCert.pem` nella directory `/etc/ipsec.d/certs`;
- copia il file `BobKey.der` nella directory `/etc/ipsec.d/private`;
- Bob riceve il file `cacertAlice.pem` da Alice e lo copia nella directory `/etc/ipsec.d/cacerts`;
- Bob avvia strongSwan col comando:

  ```
  ipsec start
  ```

- controlla che strongSwan sia in grado di trovare la chiave privata corrispondente al certificato di Bob. A questo scopo, esegue il comando:

  ```
  ipsec listcerts
  ```

  e controlla che il certificato di Bob sia elencato e contenga la linea:

  ```
  pubkey:  RSA 1024 bits, has private key
  ```

- controlla che strongSwan sia in grado di trovare il certificato della CA di Alice. A questo scopo, esegue il comando:

  ```
  ipsec listcacerts
  ```

Alice avvia la connessione configurata nel file `/etc/ipsec.conf` col comando:

```
ipsec up host2-host2
```

Analizzate col comando `ip xfrm state list` la SA generata dal demone IKE sulle macchine di Alice e Bob e rispondete alle seguenti domande:
Quante SA sono state negoziate?

Soluzione

Due, una per ogni direzione

Quale tipo di security header e di modo viene usato nelle SA? ESP o AH? Quali algoritmi sono usate? Qual è la lunghezza della chiave usata per ogni algoritmo?

Soluzione

Viene usato `esp` in `tunnel mode`. Per la cifratura si usa `aes-cbc` (con chiave a 128 bit) mentre per integrità e autenticazione si ricorre a `hmac-sha256` con chiave a 256 bit

In quale fase del protocollo IKE è usato il certifica di Alice?

Soluzione

Nella prima fase, quella per la creazione della SA IKE

In quale fase del protocollo IKE è usato il certificato di Bob?

Soluzione

Ancora nella prima fase

Alice genera traffico verso la macchina di Bob (per esempio, `icmp` e `http`) e lo analizza (per esempio con `wireshark`).

4.2 Connessione sicura gateway-to-gateway

Configurando opportunamente le regole di IPsec, e sfruttando il *tunnel mode*, è possibile creare un tunnel sicuro site-to-site.
Supponiamo di voler creare una configurazione di questo genere:

```
                        ========== ESP ==========
                        |                       |
   Network-A        Gateway-A              Gateway-B        Network-B
192.168.1.0/24 - 192.168.1.254           192.168.2.254 - 192.168.2.0/24
                  1.2.3.4 - Internet - 1.2.4.3
```

è allora necessario definire delle security policy che chiedano la creazione del tunnel per tutti gli elementi della rete A verso la rete B. Per far questo è possibile usare `ip xfrm` in tunnel mode (parametro `mode tunnel`).
In un ambiente come quello del laboratorio, dove gli host fanno tutti parte della stessa LAN, è necessario forzare le tabelle di routing in modo che il traffico sia instradato attraverso gli host richiesti, e non attraverso il normale Gateway della LAN, per simulare la presenza di molteplici LAN diverse. Una soluzione per far ciò è quella di forzare delle entry nella tabella di routing con il comando `route`. Supponendo di voler forzare una configurazione

```
   Host in A     Gateway in A     Gateway in B     Host in B
192.168.1.1 --- 192.168.3.1 ---- 192.168.3.2 --- 192.168.2.1
```

bisognerà aggiungere le seguenti entry (modificando opportunamente gli indirizzi IP con quelli delle macchine del laboratorio coinvolte):

- sulla macchina `192.168.1.1`:
  - `route add -host 192.168.2.1 gw 192.168.3.1`
- sulla macchina `192.168.3.1`
  - `route add -host 192.168.2.1 gw 192.168.3.2`
  - `route add -host 192.168.1.1 gw 192.168.1.1`
- sulla macchina `192.168.3.2`
  - `route add -host 192.168.2.1 gw 192.168.2.1`
  - `route add -host 192.168.1.1 gw 192.168.3.1`
- sulla macchina `192.168.2.1`:
  - `route add -host 192.168.1.1 gw 192.168.3.2`

Inoltre, su entrambi i Gateway sarà necessario abilitare il forwarding di pacchetti IP, opzione disabilitata di default. Un possibile modo per abilitare il forwarding su un host Linux consiste nel decommentare la linea `net.ipv4.ip_forward=1` nel file `/etc/sysctl.conf` e successivamente eseguire il comando `/sbin/sysctl -p /etc/sysctl.conf`.
Passiamo ora ad impostare le policy IPsec. Impostare le policy con `ip xfrm` è il metodo più diretto, e consiste nello specificare le policy necessarie tramite il comando `ip xfrm policy add`.
Quante SP sarà necessario specificare?

Soluzione

Sei in totale, tre per il gateway A e tre per il gateway B. Per ogni gateway infatti ne serve una per i pacchetti in ingresso (`dir in`), una per i pacchetti in uscita (`dir out`) e una per i pacchetti inoltrati (`dir fwd`).

E quante SA?

Soluzione

Bisogna impostare quattro SA, due sul gateway A (una per direzione) e due sul gateway B (una per direzione). I due gateway condividono le stesse SA, quindi di fatto se ne definiscono soltanto due che poi vengono replicate.

Scrivere i comandi richiesti (usando `ip xfrm policy`) per configurare Gateway-A e Gateway-B in modo da proteggere il traffico da un nodo nella rete `192.168.1.0/24` verso un nodo nella rete `192.168.2.0/24` per mezzo di un tunnel da `192.168.3.1` verso `192.168.3.2`, ed il traffico da un nodo nella rete `192.168.2.0/24` verso un nodo nella rete `192.168.1.0/24` creando un tunnel da `192.168.3.2` verso `192.168.3.1`. Per semplicità, forniamo il comando per la configurazione del Gateway-A:

```
ip xfrm policy add src 192.168.1.0/24 dst 192.168.2.0/24 dir out tmpl src
192.168.3.1 dst 192.168.3.2 proto esp mode tunnel level required
```

```
ip xfrm policy add src 192.168.2.0/24 dst 192.168.1.0/24 dir in tmpl src
192.168.3.2 dst 192.168.3.1 proto esp mode tunnel level required
```

```
ip xfrm policy add src 192.168.2.0/24 dst 192.168.1.0/24 dir fwd tmpl src
192.168.3.2 dst 192.168.3.1 proto esp mode tunnel level required
```

NOTA

Se si vuole simulare l'esercizio nel contesto del laboratorio bisogna ricordare che le macchine sono poste nella medesima sottorete (e.g. 192.168.1.0/24), pertanto anziché utilizzare dei range di sottorete nei campi `src` e `dst` sará necessario utilizzare gli indirizzi IP dei singoli host, A e B.

Quale comando sarà necessario per il Gateway-B? Scrivetelo a seguire:

Soluzione

```
ip xfrm policy add src 192.168.2.0/24 dst 192.168.1.0/24 dir out tmpl
src 192.168.3.2 dst 192.168.3.1 proto esp mode tunnel level required
```

```
ip xfrm policy add src 192.168.1.0/24 dst 192.168.2.0/24 dir in tmpl
src 192.168.3.1 dst 192.168.3.2 proto esp mode tunnel level required
```

```
ip xfrm policy add src 192.168.1.0/24 dst 192.168.2.0/24 dir fwd tmpl
src 192.168.3.1 dst 192.168.3.2 proto esp mode tunnel level required
```

E quali SA saranno necessarie per i due GW?

Soluzione

le SA saranno identiche su entrambi i GW:

```
ip xfrm state add src 192.168.3.2 dst 192.168.3.1 proto esp spi
0x1000 mode tunnel enc cbc(aes) 0xaa223344556677889900aabbccddeeff
ip xfrm state add src 192.168.3.1 dst 192.168.3.2 proto esp spi
0x2000 mode tunnel enc cbc(aes) 0xaa223344556677889900aabbccddeeff
```